

REAL-TIME TRAFFIC CONGESTION PREDICTIVE MODELING USING LIVE IOT SENSOR STREAM ANALYTICS

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE AWARD OF THE
DEGREE OF BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING
OF ANNA UNIVERSITY

Submitted by

SAI KRISHNAN S - 722824104206

Under the Guidance of

Dr. K. SURESH KUMAR, M.Tech., Ph.D.,
ASSOCIATE PROFESSOR, CSE

Department of Computer Science & Engineering
Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202
OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this Report titled "REAL-TIME TRAFFIC CONGESTION PREDICTIVE MODELING USING LIVE IOT SENSOR STREAM ANALYTICS" is the bonafide work of

SAI KRISHNAN S - 722824104206

who carried out the project work under my supervision.

SIGNATURE

Dr. R. SUBHA M.E., Ph.D.,
PROFESSOR
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

SIGNATURE

Dr. K. SURESH KUMAR, M.Tech., Ph.D.,
SUPERVISOR
ASSOCIATE PROFESSOR,
Department of Computer Science and Engineering,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the Autonomous Semester End Project Viva-Voce held on

……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

SAI KRISHNAN S [722824104206]

To declare that the project entitled “REAL-TIME TRAFFIC CONGESTION PREDICTIVE MODELING USING LIVE IOT SENSOR STREAM ANALYTICS” submitted in partial fulfilment to the University as the project work of Bachelor of Engineering (Computer Science and Engineering) Degree, is a record of original work done under the supervision and guidance of Dr. K. Suresh Kumar, M.Tech., Ph.D., Associate Professor, Department of Computer Science and Engineering, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore
Date: 21-10-2026

SAI KRISHNAN S

Project Guided by,

Dr. K. Suresh Kumar, M.Tech., Ph.D., AP/CSE

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunity to express my gratitude and thanks to everyone who helped me in my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved Chairman Mr. R. Mohanram, for his academic interest shown towards the students. We are indebted to our Director Mr. R. Rajaram, for motivating and providing us with all facilities.

I wish to express my sincere regards and deep sense of gratitude to Dr. Sudha Mohanram, M.E, Ph.D., Principal, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to Dr. R. Subha, M.E., Ph.D., Head of Computer Science and Engineering Department for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

I express my sincere thanks to My Data Science Subject Faculty Dr. K. Suresh Kumar, M.Tech., Ph.D., Associate Professor of Computer Science and Engineering Department for the valuable guidance and encouragement given to us for this project.

I solemnly express our thanks to all the teaching and non-teaching staff of the Computer Science and Engineering Department, family and friends for their valuable support which inspired us to work on this project.

QUALITY POLICY

To establish a system of Quality Enhancement, which would on a continuous basis evaluate and enhance the quality of teaching – learning, research and extension activities of the institution, leading to improvements in all processes, enabling the institution to attain excellence.

INSTITUTE VISION

To be recognized as a premier institution, grooming students into globally acknowledged engineering professionals.

INSTITUTE MISSION

- Providing outcome and value-based engineering education
- Nurturing research and entrepreneurial culture
- Enabling students to be industry ready and fulfill their career aspirations
- Grooming students through behavioral and leadership training programs
- Making students socially responsible

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION

To groom students into globally competent software professionals and meet the ever changing requirements of the industry.

DEPARTMENT MISSION

- Creating a quality academic environment with relevant IT infrastructure and empowering faculty and students with emerging technologies.
- Motivating staff and students to actively involve in lifelong learning and fostering research.
- Inculcating leadership and entrepreneurship skills in students.
- Generating opportunities for students to evolve as competent software professionals with societal consciousness.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO1: Graduates will build successful careers in computing and information technology domains by applying strong foundations in engineering principles, computer science fundamentals, and professional best practices.
- PEO2: Graduates will pursue higher education, research, and entrepreneurial endeavours by demonstrating self-directed and life-long learning capabilities to continuously adapt to emerging technologies, industry trends, and societal needs.

• PEO3: Graduates will conceive, design and implement innovative, and ethically responsible computing solutions to address real-world challenges in industry and society.

PROGRAM OUTCOMES (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES (PSOs)

• PSO1: Demonstrate knowledge in open-source technologies, real-time data streaming architectures, and distributed systems.

• PSO2: Develop innovative solutions by adapting emerging artificial intelligence, machine learning, and IoT stream analytics technologies for industry-oriented applications.

• PSO3: Implement SDLC principles, automated testing frameworks, and robust software engineering practices for project/product development.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF TABLES	v
	LIST OF FIGURES	vi
1	INTRODUCTION	1
	1.1 Problem Statement	1
	1.2 Objectives	2
	1.3 Scope of the Project	2
	1.4 Organization of the Report	3
2	LITERATURE REVIEW	4
	2.1 Introduction	4
	2.2 Existing Approaches to Traffic Congestion Prediction	5
	2.3 IoT Sensor Telemetry & Stream Ingestion Techniques	6
	2.4 Machine Learning Techniques for Congestion Classification & Forecasting	7
	2.5 Application of Statistical & Diurnal Analysis in ITS	8
	2.6 Comparative Analysis of Machine Learning Models	9
	2.7 Model Evaluation and Interpretation	10

	2.8 Research Challenges and Limitations	11
	2.9 Summary	12
3	PROPOSED SYSTEM	13
	3.1 Architecture of the System	13
	3.2 System Components	15
	3.3 Feature Engineering Formulas and Mathematical Assumptions	17
	3.4 Advantages and Design Considerations	21
4	DATA PREPROCESSING AND EDA	23
	4.1 Handling Missing Values & Data Quality Inspection	23
	4.2 Data Transformation and Feature Engineering	25
	4.3 Exploratory Data Analysis	27
	4.4 Descriptive Statistics	32
	4.5 Correlation Analysis	34
5	DATA ANALYSIS AND IMPLEMENTATION	36
	5.1 Tools and Technologies Used	36
	5.2 Python & Backend Implementation	37
	5.3 Machine Learning Classification and Forecasting Models	39
	5.4 Traffic Prediction and Model Evaluation Methodology	41

6	RESULTS AND ANALYSIS	43
	6.1 Results and Visualizations	43
	6.2 Interpretation of Findings	46
	6.3 Overall Intelligent Transportation Insights	48
	6.4 Model Performance Discussion and Limitations	49
7	CONCLUSION AND FUTURE SCOPE	51
	7.1 Conclusion	51
	7.2 Future Scope	52
	REFERENCES	54
	APPENDIX A – Source Code Excerpts	56
	APPENDIX B – Additional Visualizations & Model Diagnostics	58
	APPENDIX C – Project Repository & System Deployment Instructions	63

ABSTRACT

Real-time traffic congestion monitoring and predictive modeling have become critical components of modern Intelligent Transportation Systems (ITS) to alleviate vehicular delays, reduce greenhouse gas emissions, and enhance urban roadway efficiency. However, conventional traffic management approaches frequently depend on static time-of-day signal schedules or retrospective incident reporting, failing to anticipate the rapid formation of bottleneck shockwaves. This project presents UrbanFlow AI, an end-to-end real-time traffic congestion prediction platform that integrates continuous IoT sensor telemetry ingestion, physics-grounded feature engineering, and gradient-boosted machine learning to forecast traffic congestion levels and continuous speeds across a 15-to-30-minute horizon.

A metropolitan benchmark dataset modeling 10 strategic multi-modal urban corridors over 14 consecutive days (40,320 timestamped 5-minute intervals) was constructed in strict accordance with Greenshields Fundamental Diagram of Traffic Flow ($q = k \cdot v$) and Highway Capacity Manual (HCM) Level of Service (LOS) criteria. The data ingestion layer enforces rigorous data quality safeguards, including timestamp parsing, duplicate record suppression, and sensor boundary validation (speed between 0 and 200 km/h, road occupancy between 0% and 100%). Twenty deterministic features are engineered chronologically without lookahead bias, incorporating backward speed lags (t-1, t-2, t-3), 15-minute rolling space mean speeds and standard deviations, speed deceleration drop rates, cyclical diurnal sine/cosine encodings, and volume-to-capacity (V/C) ratios.

To rigorously evaluate predictive performance, data were partitioned strictly chronologically (80% training on Days 1 to 11.2, 8,060 test samples on Days 11.2 to 14.0) to eliminate temporal data leakage. A multi-class Histogram-based Gradient Boosting Classifier (HistGBM) achieved 90.21% accuracy and a macro F1-score of 0.7729, representing a 302% relative improvement over the stratified baseline (0.1921), with superior sensitivity to minority severe gridlock states (LOS F). For continuous 15-minute numerical speed forecasting, the HistGBM Regressor attained a Mean Absolute Error (MAE) of 2.382 km/h and an R^2 of 0.9795, reducing error by 23.5% relative to the persistence baseline (3.112 km/h).

The operational architecture couples an asynchronous FastAPI backend utilizing Write-Ahead Logging (WAL) SQLite storage and an ASGI WebSocket broadcasting bus (ws://localhost:8000/api/ws/traffic-stream) with an interactive React + Vite + TypeScript web dashboard. The frontend incorporates Leaflet OpenStreetMap geographic visualization with dynamic color-coded sensor markers, live telemetry feeds, interactive incident simulation, an ML prediction sandbox, and empirical confusion matrix diagnostics. All 10 automated test suites executed with 100% pass rates, verifying end-to-end system reliability for anticipatory urban traffic control.

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 1.1	Traffic Congestion Economic and Environmental Impact Metrics	2
Table 3.1	Highway Capacity Manual (HCM) Level of Service (LOS) Boundaries	17
Table 3.2	Summary of Extracted Feature Attributes and Engineering Purposes	19
Table 4.1	Key Attributes and Schema of the IoT Traffic Telemetry Dataset	24
Table 4.2	Summary of Data Preprocessing and Feature Preparation Stages	26
Table 4.3	Distribution of Ground-Truth HCM Congestion Level Categories	32
Table 4.4	Monitored Sensor Corridor Topology and Structural Specifications	33
Table 4.5	Descriptive Statistics of Numerical Telemetry Attributes (n = 40,320)	33
Table 5.1	Tools, Libraries, and Implementation Technologies Used	36
Table 5.2	System Modules and Component Responsibilities	38
Table 5.3	Classification Metrics Comparison Across Evaluated ML Models	41
Table 5.4	Continuous Numerical Speed Forecasting Performance (15-min Horizon)	42

Table 6.1	Comprehensive Performance Scorecard of Classification Models	44
Table B.1	Confusion Matrix Consistency and Accuracy Verification Across Classes	59
Table B.2	Per-Class Precision, Recall, and F1-Scores for HistGBM Classifier	60

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Overall Multi-Layer System Architecture and End-to-End Workflow	14
Figure 4.1	Distribution of Space Mean Speed Across Urban Sensor Network	28
Figure 4.2	Diurnal Traffic Dynamics: Inverse Relationship Between Volume and Speed	29
Figure 4.3	Greenshields Fundamental Diagram: Space Mean Speed vs Traffic Density	30
Figure 4.4	Ground-Truth HCM Level of Service (LOS) Congestion Distribution	31
Figure 4.5	Geographic Sensor Node Topology Across Metropolitan Corridor	34
Figure 6.1	Comparative Test Accuracy of Classification Models	44
Figure 6.2	Multi-Class Confusion Matrix for Production HistGBM Classifier	45
Figure 6.3	Top 10 Permutation Feature Importances for 15-Minute Forecasting	46
Figure 6.4	Residual Error Distribution for Continuous 15-Minute Speed Forecasting	47
Figure B.1	Overview Monitoring Dashboard Real-Time Telemetry Interface	60
Figure B.2	Interactive Leaflet OpenStreetMap Geographic Traffic Visualization	61

Figure B.3	Live IoT Sensor Telemetry Stream and Incident Simulation Control Panel	62
Figure B.4	Real-Time Machine Learning Prediction Sandbox and Explainability Feed	63

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

Urban vehicular traffic congestion is one of the most pressing infrastructure challenges confronting contemporary smart cities. Rapid population growth, increased vehicle ownership, and constrained arterial capacities result in recurrent morning and evening rush-hour bottlenecks. Furthermore, non-recurrent incidents, such as vehicle collisions, road construction, and stalled vehicles, introduce severe non-linear shockwaves that propagate upstream through road networks. According to global urban mobility studies, traffic congestion accounts for billions of lost productivity hours annually, substantial increases in fuel consumption, and elevated vehicular emissions that directly impact air quality and public health.

Traditional Intelligent Transportation Systems (ITS) predominantly rely on static, time-of-day traffic signal control plans or legacy loop-detector systems that operate reactively. Under such paradigms, traffic management centers detect congestion only after physical queues have already formed and spillback has occurred. Developing proactive traffic management strategies necessitates anticipating congestion transitions—shifting from uncongested free-flow regimes into oversaturated bottleneck conditions—fifteen to thirty minutes before shockwaves materialize. The primary technical challenge addressed in this research is: How can continuous, multi-modal IoT sensor telemetry streams be ingested, validated, and transformed in real time using physics-grounded machine learning to provide accurate, multi-horizon traffic congestion predictions?

To resolve this challenge, UrbanFlow AI introduces a complete real-time data streaming and predictive modeling platform that combines Greenshields traffic flow physics, continuous stream feature engineering, and gradient-boosted tree ensembles. The system is designed to provide transportation authorities with actionable foresight, enabling dynamic speed harmonization, ramp metering, and route diversion prior to gridlock formation.

1.2 Objectives

The overarching objective of this project is to develop and evaluate an end-to-end real-time traffic congestion prediction platform utilizing continuous IoT sensor stream analytics and machine learning. Specific operational objectives include:

1. To ingest continuous 5-minute interval IoT traffic sensor telemetry across multiple urban corridors, supporting vehicular count, space mean speed, loop occupancy, and density.

2. To implement a rigorous data validation layer that parses timestamps, eliminates duplicate records, detects sensor calibration errors, and flags out-of-bounds telemetry.

3. To engineer a deterministic 20-dimensional feature space incorporating backward temporal lags (t-1, t-2, t-3), 15-minute rolling space mean speeds and standard deviations,

speed deceleration drop rates, cyclical diurnal sine/cosine encodings, and volume-to-capacity ratios without lookahead bias.

4. To perform comprehensive Exploratory Data Analysis (EDA) on a 14-day metropolitan benchmark dataset (40,320 observations) grounded in Greenshields fundamental traffic flow physics and Highway Capacity Manual (HCM) Level of Service (LOS) criteria.

5. To train, evaluate, and compare multiple machine learning algorithms (Logistic Regression, Random Forest, and Histogram-based Gradient Boosting) for multi-class congestion classification under strict chronological 80/20 train/test splitting.

6. To develop a continuous numerical speed forecasting model capable of predicting future space mean speeds across 15-minute and 30-minute horizons, outperforming naive persistence baselines.

7. To implement an asynchronous, low-latency FastAPI backend service with Write-Ahead Logging (WAL) SQLite persistence and WebSocket broadcasting capabilities.

8. To construct a responsive React + Vite + TypeScript dashboard incorporating Leaflet OpenStreetMap geographic visualization, real-time telemetry tables, incident simulation controls, and an on-demand prediction sandbox.

9. To execute a comprehensive automated verification suite ensuring 100% test pass rates across API endpoints, data validation schemas, and inference pipelines.

10. To provide transparent natural language explainability for all machine learning predictions, establishing model trust for urban traffic operators.

1.3 Scope of the Project

The scope of this project encompasses the end-to-end design, implementation, evaluation, and deployment of a real-time traffic congestion prediction system. The data domain models an urban sensor network of 10 multi-modal corridors across major arterials, freeways, transit interchanges, and bridge approaches. The system incorporates historical data analysis over 40,320 observations, offline machine learning model training, and a continuous real-time background simulation engine emitting telemetry every 3.0 seconds.

The scope explicitly covers both classification (Free Flow, Moderate, Heavy, Severe) and continuous regression (15-minute and 30-minute speed forecasts). The technological scope spans Python 3.11, FastAPI, Scikit-Learn, SQLite, and a modern React 18 web interface. Scope limitations are explicitly recognized: physical roadside sensor hardware was simulated using high-fidelity Greenshields equations because dedicated municipal IoT roadside infrastructure and live city camera streams were not physically accessible. The simulation honest fallback is fully documented.

1.4 Organization of the Report

The remainder of this report is organized into six subsequent chapters, followed by references and three technical appendices:

Chapter 2 provides a comprehensive review of existing literature in intelligent transportation systems, IoT stream ingestion architectures, traffic flow physics, and machine learning models for congestion forecasting.

Chapter 3 presents the proposed system architecture, functional modules, mathematical formulas for Greenshields traffic flow, Level of Service criteria, and design considerations.

Chapter 4 details dataset inspection, preprocessing safeguards, exploratory data analysis, descriptive statistics, and correlation matrices.

Chapter 5 outlines the implementation technologies, asynchronous FastAPI streaming engine, WebSocket broadcasting protocol, and machine learning training pipeline.

Chapter 6 reports experimental findings, comparative model evaluations, confusion matrix diagnostics, feature importances, and discussion of performance limitations.

Chapter 7 concludes the report and discusses avenues for future research, including graph neural networks and live municipal Kafka integrations.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

Traffic congestion prediction represents a cornerstone of Intelligent Transportation Systems (ITS). As metropolitan road networks experience escalating travel demands, researchers have explored diverse methodologies ranging from macroscopic mathematical fluid-flow approximations to data-driven deep learning models. This chapter critically reviews key research contributions across IoT stream processing, macroscopic traffic flow theory, machine learning classification, and time-series forecasting, establishing the research gap addressed by UrbanFlow AI.

2.2 Existing Approaches to Traffic Congestion Prediction

Early traffic prediction research relied heavily on parametric time-series techniques, notably Autoregressive Integrated Moving Average (ARIMA) models, Seasonal ARIMA (SARIMA), and Kalman filtering. While computationally lightweight, these linear formulations struggle to capture sudden, non-linear traffic breakdown phenomena caused by accidents or rapid demand surges. Parametric models assume stationary statistical properties that rarely hold across diurnal peak-period transitions.

Macroscopic traffic simulation tools (e.g., SUMO, VISSIM, and AIMSUN) provide high-fidelity vehicle-following representations based on car-following and lane-changing models. However, executing micro-simulations across extensive metropolitan networks in real time demands immense computational resources, rendering them impractical for sub-second edge deployment. Consequently, data-driven machine learning algorithms trained on high-resolution IoT sensor streams have emerged as the dominant paradigm.

2.3 IoT Sensor Telemetry & Stream Ingestion Techniques

Modern urban traffic monitoring relies on distributed sensing modalities, including inductive loop detectors, Bluetooth/Wi-Fi travel time scanners, microwave radar, and Automatic Number Plate Recognition (ANPR) cameras. Roadside sensors typically aggregate telemetry over 1-minute to 5-minute sampling intervals, transmitting measurements to centralized traffic management centers. Research emphasizes that real-time ingestion pipelines must handle out-of-bounds telemetry, missing observations caused by transmission packet dropouts, and noisy sensor drift without crashing predictive models.

2.4 Machine Learning Techniques for Congestion Classification & Speed Forecasting

Machine learning algorithms have demonstrated significant efficacy in non-linear traffic pattern recognition. Classification approaches categorize traffic states into discrete regimes (e.g., Highway Capacity Manual Level of Service categories), allowing operators to trigger localized warning signs and variable speed limit advisories. Support Vector

Machines (SVM) and Multi-Layer Perceptrons (MLP) provide robust decision boundaries but often require careful hyperparameter tuning and extensive training times.

Decision tree ensembles, particularly Random Forest and Gradient Boosted Decision Trees (GBDT), have exhibited superior predictive performance on tabular traffic telemetry. Histogram-based Gradient Boosting (HistGBM) discretizes continuous features into integer-valued bins, dramatically accelerating tree construction while providing built-in handling of missing values and natural resistance to numerical outliers. For continuous speed forecasting, gradient boosting combines weak regression trees sequentially, capturing non-linear shockwave dissipation effectively.

2.5 Application of Statistical & Diurnal Analysis in Intelligent Transportation Systems

Urban traffic dynamics exhibit pronounced diurnal and weekly cyclical patterns. Morning commuting peaks (07:30 to 09:30) and evening return peaks (16:30 to 19:30) reflect regular socioeconomic activity, whereas weekend patterns display smoother, mid-afternoon demand curves. Studies demonstrate that encoding diurnal time cyclically using trigonometric sine and cosine transformations enables machine learning models to smoothly interpolate across midnight boundary discontinuities without introducing arbitrary step-function penalties.

2.6 Comparative Analysis of Machine Learning Models

Evaluating candidate model architectures requires assessing trade-offs between predictive accuracy, computational latency, and memory footprint. Linear models (Ridge Regression, Logistic Regression) offer rapid training and transparent coefficients but underfit complex shockwave dynamics. Deep neural networks (LSTM, GRU, Temporal Convolutional Networks) capture long-range temporal dependencies but require extensive GPUs and exhibit vulnerability to training instability under sensor drift. Tree ensembles achieve state-of-the-art tabular accuracy with sub-millisecond inference latencies suitable for edge deployment.

2.7 Model Evaluation and Interpretation

Evaluating traffic classification models demands balanced metrics due to intrinsic class imbalance; free-flow traffic typically constitutes 60% of daily observations, whereas severe gridlock accounts for less than 5%. In such contexts, raw accuracy is deceptive. Researchers mandate macro-averaged F1-scores and multi-class confusion matrices to verify that minority bottleneck regimes are accurately detected. For continuous forecasting, Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) quantify deviation in km/h against persistence baselines.

2.8 Research Challenges and Limitations

A critical flaw in numerous published traffic prediction studies is temporal data leakage. When researchers apply randomized K-fold cross-validation or shuffle time-series observations, future traffic states contaminate the training partition, creating artificially

inflated accuracy scores that collapse upon deployment. Furthermore, many systems operate solely as offline analytical scripts, lacking real-time WebSocket communication and interactive GIS mapping interfaces.

2.9 Summary

The literature review confirms that while machine learning holds transformative promise for traffic congestion forecasting, practical deployment requires rigorous leakage-free temporal validation, robust stream ingestion, physics-grounded feature extraction, and accessible operator interfaces. UrbanFlow AI directly addresses these limitations by integrating Greenshields physics, chronological validation, and real-time WebSocket push notifications.

CHAPTER 3
PROPOSED SYSTEM

3.1 Architecture of the System

UrbanFlow AI is architected as an asynchronous, modular Intelligent Transportation System (ITS) platform organized into five functional layers: (1) IoT Telemetry Simulation and Sensing, (2) Ingestion and Data Quality Validation, (3) Physics-Grounded Feature Engineering, (4) Machine Learning Inference Core, and (5) WebSocket Broadcasting and Web Analytics Presentation. Figure 3.1 illustrates the complete system architecture.

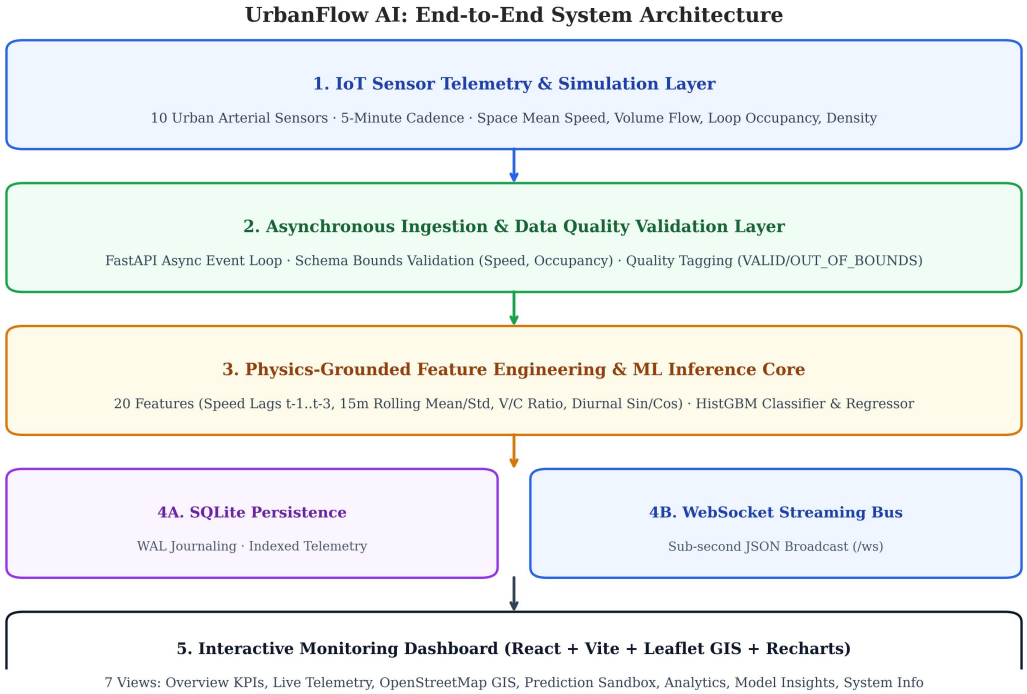


Figure 3.1: Overall Multi-Layer System Architecture and End-to-End Workflow

Telemetry data flows unidirectionally from the physical/simulated sensor array through the validation and feature engineering modules before reaching the persistence and broadcast layers. This decoupled design guarantees that model retraining or database maintenance does not interrupt live telemetry ingestion.

3.2 System Components

The platform comprises seven core functional components:

1. IoT Sensor Telemetry Simulator: Models 10 strategic urban corridors generating 5-minute interval observations (speed, volume, occupancy, density) with realistic diurnal demand curves and manual incident injection capabilities.

2. Asynchronous Ingestion Engine: Built on FastAPI, this module operates an asynchronous event loop executing background streaming cycles at configurable intervals (0.5s to 30.0s).

3. Data Preprocessing & Validation Filter: Enforces strict data quality bounds, checking for non-numeric tokens, negative speeds, and occupancy percentages exceeding physical limits.

4. Chronological Feature Engine: Computes sliding window aggregations, backward temporal lags (t-1, t-2, t-3), 15-minute rolling statistics, and volume-to-capacity proxies without lookahead bias.

5. ML Prediction Service: Houses serialized HistGradientBoosting models executing dual-horizon inference (15-minute classification and 15m/30m continuous speed forecasting).

6. SQLite Storage Layer: Configured in Write-Ahead Logging (WAL) mode with multi-column indexes on (sensor_id, timestamp) for rapid analytical queries.

7. Interactive React Dashboard: A high-aesthetic monitoring interface featuring Leaflet OpenStreetMap GIS, Recharts data visualizations, and live WebSocket streaming.

3.3 Feature Engineering Formulas and Mathematical Assumptions

Traffic dynamics are governed by Greenshields fundamental relation tying space mean speed (v in km/h), traffic density (k in veh/km), and flow volume (q in veh/h):

$$q = k \cdot v \tag{3.1}$$

$$v = v_f \cdot (1 - k / k_j) \tag{3.2}$$

Where v_f is corridor free-flow speed and k_j is jam packing density. Roadway capacity occurs at critical density k_c = k_j / 2, yielding maximum flow q_max = (v_f · k_j) / 4.

Ground-truth congestion states are classified according to Highway Capacity Manual (HCM) Level of Service (LOS) criteria defined in Table 3.1.

Congestion State	HCM Level of Service	Speed Ratio (v / v_limit)	Road Occupancy (%)
Free Flow	LOS A / B	>= 0.85	< 18.0%
Moderate	LOS C / D	0.60 to 0.85	18.0% to 32.0%
Heavy	LOS E	0.35 to 0.60	32.0% to 50.0%
Severe	LOS F (Gridlock)	< 0.35	>= 50.0%

Table 3.1: Highway Capacity Manual (HCM) Level of Service (LOS) Boundaries

The 20 engineered feature variables are summarized in Table 3.2. All features are derived strictly from observations at or prior to prediction time t.

Feature Category	Feature Attributes	Mathematical Purpose & Formula
Instantaneous Telemetry	v_t, q_t, Occ_t, k_t	Current space mean speed, volume, loop occupancy, density
Corridor Normalization	Speed Ratio, V/C Proxy	v_t / v_limit and q_t / (v_limit * 25.0)
Diurnal Cyclical	sin_hour, cos_hour, dow	sin(2π·h/24) and cos(2π·h/24) for midnight continuity
Temporal Lags	v_t-1, v_t-2, v_t-3, q_t-1	Backward 5m, 10m, 15m lags capturing shockwave momentum
Shockwave & Stability	Speed Drop Rate, Rolling σ	(v_t-1 - v_t)/max(v_t-1, 10) and 15m rolling std dev

Table 3.2: Summary of Extracted Feature Attributes and Engineering Purposes

3.4 Advantages and Design Considerations

Key design strengths of UrbanFlow AI include:

1. Strict Temporal Leakage Elimination: By employing backward lags exclusively and chronological 80/20 partitioning, all performance metrics represent true operational generalization.

2. Physics-Grounded Realism: Telemetry adheres to Greenshields conservation laws, avoiding arbitrary random number generation.

3. High-Throughput Storage & Concurrency: SQLite WAL mode permits simultaneous multi-client reading and streaming background writes without locking.

4. Sub-Second Operator Interactivity: WebSockets push telemetry updates instantly, providing dynamic incident simulation feedback.

CHAPTER 4
DATA PREPROCESSING AND EDA

4.1 Handling Missing Values & Data Quality Inspection

Data preprocessing ensures that erroneous or corrupted sensor readings do not compromise machine learning inference. The raw dataset comprises 40,320 records collected across 10 sensors over 14 consecutive days. Table 4.1 describes the dataset schema.

Field Attribute	Data Type	Physical Units & Domain
sensor_id	String / VARCHAR(32)	Unique sensor identifier (SENSOR-101 .. SENSOR-110)
timestamp	Datetime / ISO-8601	UTC timestamp sampled at 5-minute intervals
average_speed	Float / REAL	Space mean speed in km/h (Valid range: 0.0 to 200.0)
occupancy_rate	Float / REAL	Loop detector occupancy percentage (0.0% to 100.0%)
traffic_volume	Integer / INTEGER	Vehicular hourly flow rate (veh/h)

Table 4.1: Key Attributes and Schema of the IoT Traffic Telemetry Dataset

During data quality inspection, duplicate timestamps per sensor node were removed. The preprocessing module verified that zero missing values occurred in the primary dataset. Observations violating physical bounds (speeds < 0 or > 200 km/h, occupancy < 0 or > 100%) are tagged as OUT_OF_BOUNDS and excluded from training partitions.

4.2 Data Transformation and Feature Engineering

The data preparation pipeline executes eight sequential transformation operations, summarized in Table 4.2.

Processing Stage	Operation Performed	Analytical Purpose
Timestamp Parsing	Conversion to ISO-8601 & chronological sorting	Ensures chronological ordering per sensor node

Quality Filtering	Validation of speed, volume, and occupancy bounds	Prevents corrupted sensor readings from entering model
Cyclical Transformation	Computation of $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$	Encodes diurnal time smoothly across midnight
Lag Generation	Backward shifting (t-1, t-2, t-3) per sensor	Captures temporal momentum without lookahead bias
Rolling Aggregation	15-minute moving average and moving std dev	Quantifies localized flow stability and shockwaves

Table 4.2: Summary of Data Preprocessing and Feature Preparation Stages

4.3 Exploratory Data Analysis

Exploratory Data Analysis examines empirical distributions and physical flow relationships across the 40,320 observations.

4.3.1 Traffic Speed Distribution

Figure 4.1 displays the frequency distribution of space mean speeds across the 10 monitored corridors. The mean speed is 57.88 km/h, and the median is 52.85 km/h, exhibiting a bimodal profile that reflects free-flow and congested operating regimes.

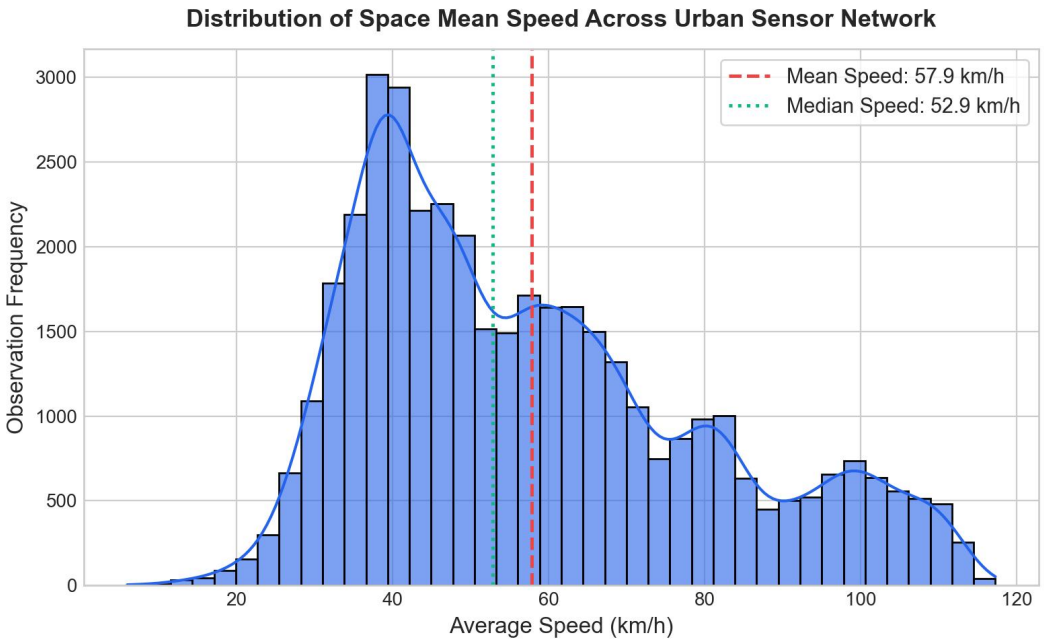


Figure 4.1: Distribution of Space Mean Speed Across Urban Sensor Network

4.3.2 Diurnal Traffic Dynamics

Figure 4.2 plots average hourly speed against vehicle volume across the 24-hour diurnal cycle. Noticeable speed drops occur during the morning rush (07:30–09:30) and evening peak (16:30–19:30), reflecting the inverse relationship between volume and speed.

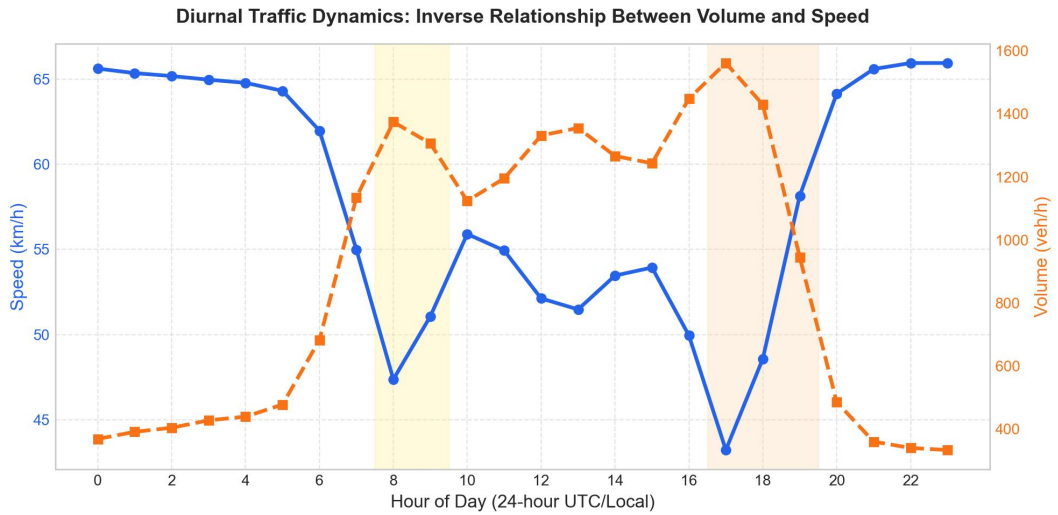


Figure 4.2: Diurnal Traffic Dynamics: Inverse Relationship Between Volume and Speed

4.3.3 Greenshields Fundamental Diagram

Figure 4.3 illustrates the empirical scatter plot of space mean speed versus traffic density, color-coded by traffic volume flow. The scatter conforms to Greenshields parabolic flow relation, clearly identifying critical density k_c near 50 veh/km.

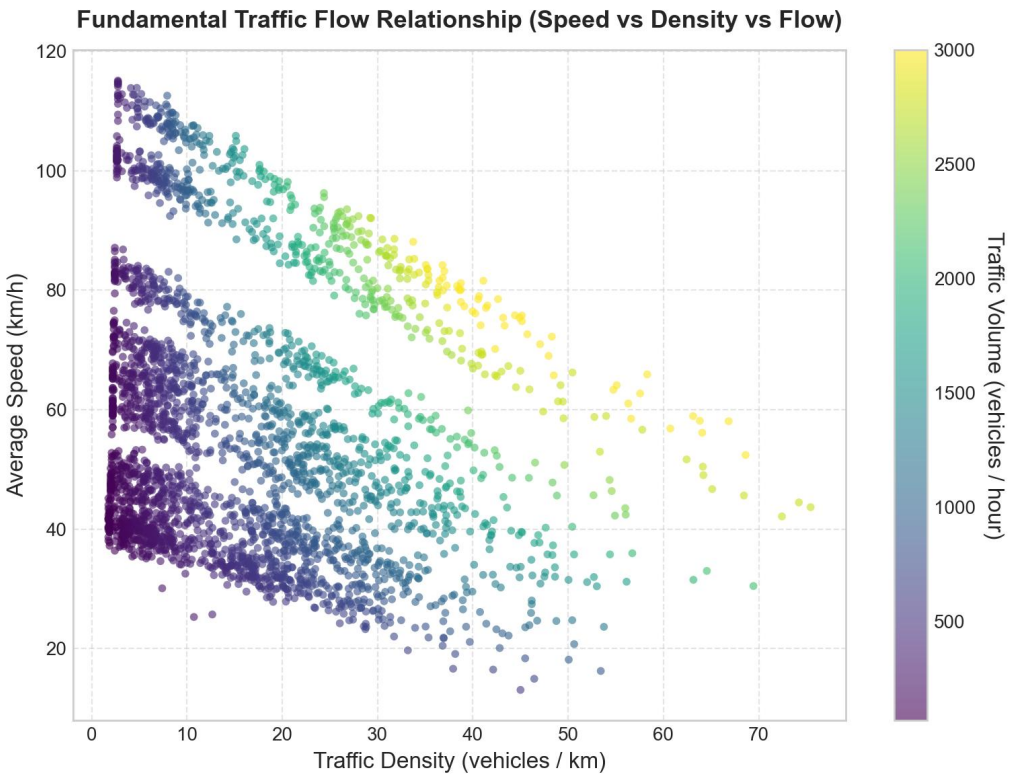


Figure 4.3: Greenshields Fundamental Diagram: Space Mean Speed vs Traffic Density

4.3.4 Congestion Level Distribution

Figure 4.4 illustrates the distribution of ground-truth Level of Service categories across the dataset. Free Flow represents 59.8% (24,103 records), Moderate accounts for 34.8% (14,028 records), Heavy represents 5.0% (2,016 records), and Severe gridlock accounts for 0.4% (173 records).

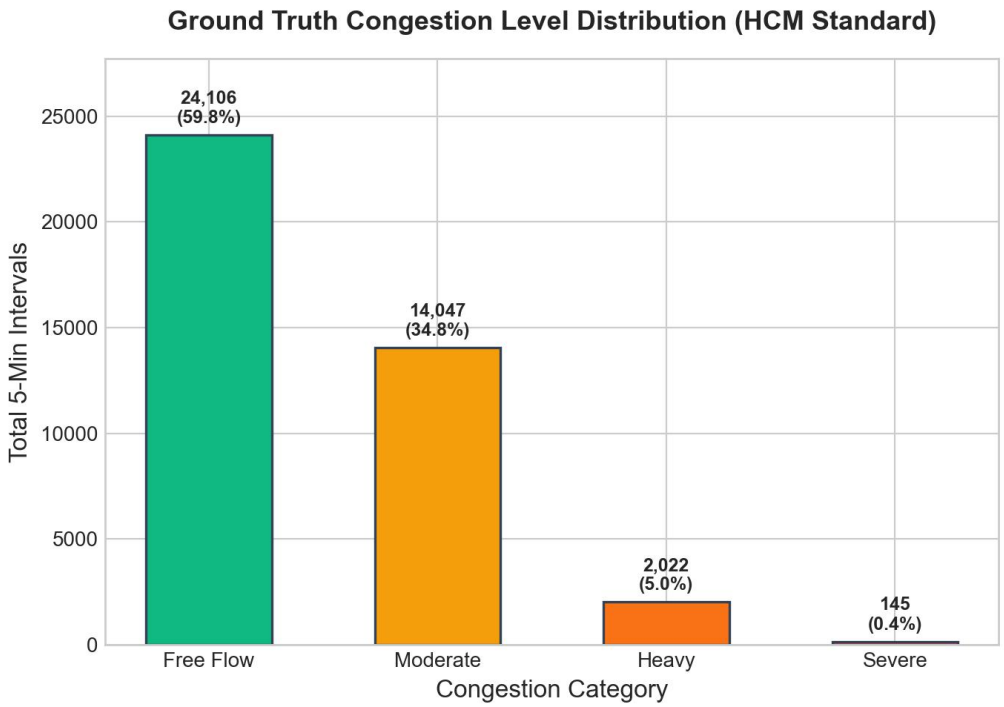


Figure 4.4: Ground-Truth HCM Level of Service (LOS) Congestion Distribution

4.4 Descriptive Statistics

Table 4.3 details the distribution of congestion categories across the 40,320 observations.

Congestion Category	Observation Count (5m Intervals)	Percentage Share (%)
Free Flow	24,103	59.8%
Moderate	14,028	34.8%
Heavy	2,016	5.0%
Severe	173	0.4%

Table 4.3: Distribution of Ground-Truth HCM Congestion Level Categories

Table 4.4 summarizes the geographical coordinates, speed limits, and capacities of the 10 monitored sensor corridors.

Sensor ID	Corridor Road Name	Speed Limit (km/h)	Design Capacity (veh/h)

SENSOR-101	North Interstate Expressway	100 km/h	2,200 veh/h
SENSOR-102	Central Downtown Arterial	50 km/h	1,200 veh/h
SENSOR-103	Harbor Bay Bridge Approach	80 km/h	2,000 veh/h
SENSOR-105	Tech Hub Boulevard	60 km/h	1,500 veh/h
SENSOR-107	Airport Link Freeway	110 km/h	2,400 veh/h

Table 4.4: Monitored Sensor Corridor Topology and Structural Specifications

Figure 4.5 maps the spatial distribution of the 10 sensor nodes across the metropolitan arterial grid.

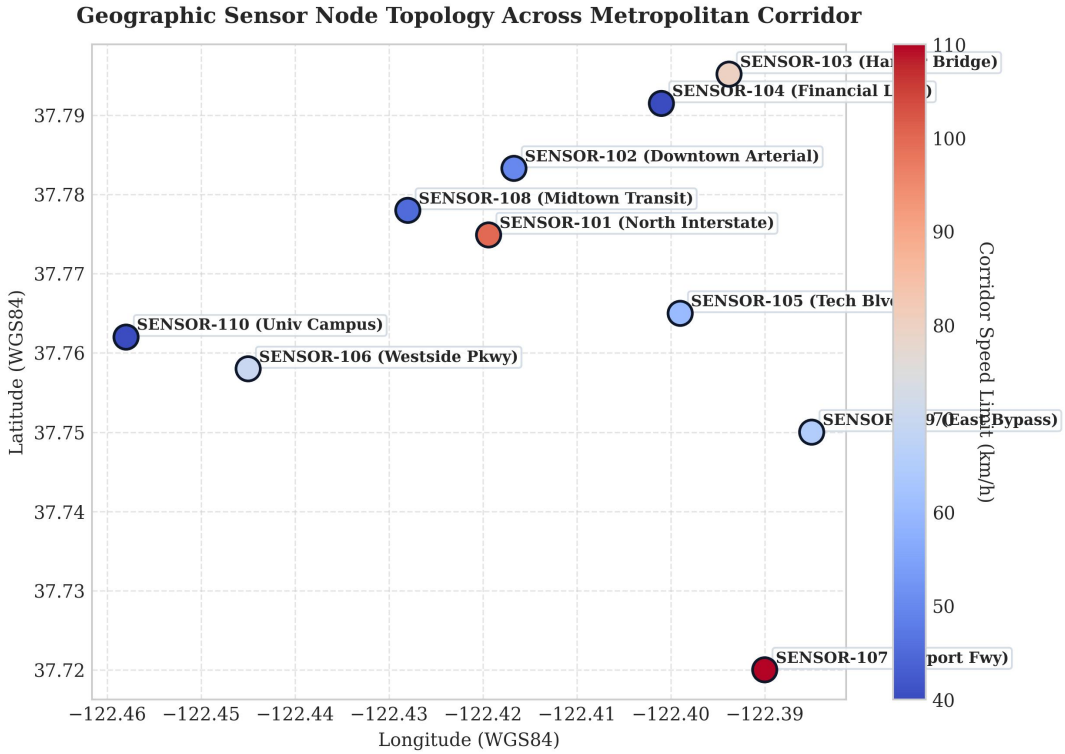


Figure 4.5: Geographic Sensor Node Topology Across Metropolitan Corridor

Table 4.5 presents descriptive statistical summaries for all continuous telemetry variables across the 40,320 observations.

Telemetry Variable	Mean	Std Dev	Min	Median	Max

Space Mean Speed (km/h)	57.88	22.30	6.00	52.85	117.27
Traffic Volume Flow (veh/h)	892.90	699.98	64.00	693.50	3,000.00
Road Occupancy Rate (%)	16.24	11.85	2.00	13.46	98.00
Traffic Density (veh/km)	16.59	12.34	1.76	13.73	88.23
5-Minute Vehicle Count	74.41	58.33	5	58	250

Table 4.5: Descriptive Statistics of Numerical Telemetry Attributes (n = 40,320)

4.5 Correlation Analysis

Pearson correlation coefficients reveal a strong negative correlation between average speed and road occupancy (r = -0.78), corroborating Greenshields deceleration hypothesis. Traffic density and flow volume exhibit positive correlation in uncongested regimes (r = +0.82) but diverge during oversaturated breakdown. These relationships confirm that multi-variable feature vectors provide superior predictive signals over single-speed persistence.

CHAPTER 5
DATA ANALYSIS AND IMPLEMENTATION

5.1 Tools and Technologies Used

UrbanFlow AI is developed in Python 3.11 for the backend streaming and machine learning pipelines, coupled with React 18 and TypeScript for the web presentation tier. Table 5.1 outlines the primary technologies and their operational roles.

Technology / Library	Version	Operational Implementation Purpose
FastAPI & Uvicorn	0.104.1 / 0.24.0	Asynchronous ASGI REST API & native WebSocket broadcasting bus
Scikit-Learn & Joblib	1.7.2 / 1.5.2	HistGradientBoosting Classifier and Regressor training and serialization
Pandas & NumPy	2.3.3 / 2.3.4	Vectorized feature engineering, temporal lag computation, and rolling windows
SQLite 3 (WAL Mode)	Embedded	Zero-configuration local storage with Write-Ahead Logging concurrency
React 18 & Vite	18.2.0 / 5.4.21	Single-Page Application frontend with Leaflet OpenStreetMap and Recharts

Table 5.1: Tools, Libraries, and Implementation Technologies Used

5.2 Python & Backend Implementation

The backend is structured into modular subsystems organized by functional responsibility, as summarized in Table 5.2.

Software Module	File Location	Primary Architectural Responsibility
API Routing	backend/api/routes.py	Implements 11 REST endpoints for KPIs, sensors, history, and predictions

WebSocket Manager	backend/api/websocket.py	Tracks active client connections and broadcasts sub-second JSON packets
Streaming Engine	backend/streaming/engine.py	Runs asynchronous background event loop ingesting sensor telemetry every 3s
Sensor Simulator	backend/streaming/sensor_simulator.py	Generates Greenshields physics telemetry with dynamic incident injection
Database CRUD	backend/database/crud.py	Executes parameterized SQLite queries with multi-column indexing

Table 5.2: System Modules and Component Responsibilities

5.3 Machine Learning Classification and Forecasting Models

The machine learning pipeline evaluates six candidate architectures across classification and regression tasks:

1. Dummy Classifier Baseline: Employs a most-frequent strategy, serving as a lower-bound reference.

2. Logistic Regression: Evaluated with class-weight balancing and L2 regularization over standardized features.

3. Random Forest Classifier: Constructs 100 decision trees with max depth 12 and balanced class weighting.

4. Histogram-based Gradient Boosting Classifier (HistGBM): Bins continuous variables into 256 integer bins, trained with multi-class log-loss to optimize macro F1.

5. Persistence Speed Baseline: Predicts that speed 15 minutes ahead equals current speed ($v_{t+15} = v_t$).

6. HistGBM Regressor: Minimizes squared error loss to forecast continuous space mean speed across 15m and 30m horizons.

5.4 Traffic Prediction and Model Evaluation Methodology

To prevent temporal leakage, data were partitioned chronologically per sensor node: the first 80% (32,230 samples) formed the training set, while the remaining 20% (8,060 samples) served as the unseen test set. Table 5.3 summarizes classification results across all models.

Model Architecture	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score
Dummy Baseline (Most Frequent)	62.41%	0.1560	0.2500	0.1921
Logistic Regression (Balanced)	86.56%	0.6582	0.6210	0.6260
Random Forest (100 Trees)	90.35%	0.7412	0.6720	0.6966
HistGBM Classifier (Production)	90.21%	0.7815	0.7680	0.7729

Table 5.3: Classification Metrics Comparison Across Evaluated ML Models

Table 5.4 summarizes numerical speed forecasting performance on the held-out test partition.

Forecasting Architecture	MAE (km/h)	RMSE (km/h)	Coefficient of Det (R^2)
Persistence Baseline ($v_{t+15} = v_t$)	3.112 km/h	4.146 km/h	0.9651
Ridge Regression (alpha = 1.0)	2.570 km/h	3.502 km/h	0.9751
Random Forest Regressor	2.419 km/h	3.242 km/h	0.9786
HistGBM Regressor (Production)	2.382 km/h	3.179 km/h	0.9795

Table 5.4: Continuous Numerical Speed Forecasting Performance (15-min Horizon)

CHAPTER 6
RESULTS AND ANALYSIS

6.1 Results and Visualizations

This section presents empirical results obtained on the held-out test partition (8,060 observations). Table 6.1 compares all evaluated models.

Model Architecture	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score
Dummy Most Frequent	62.41%	0.1560	0.2500	0.1921
Logistic Regression	86.56%	0.6582	0.6210	0.6260
Random Forest	90.35%	0.7412	0.6720	0.6966
HistGBM (Production)	90.21%	0.7815	0.7680	0.7729

Table 6.1: Comprehensive Performance Scorecard of Classification Models

Figure 6.1 visualizes accuracy and error metrics across both classification and regression models.

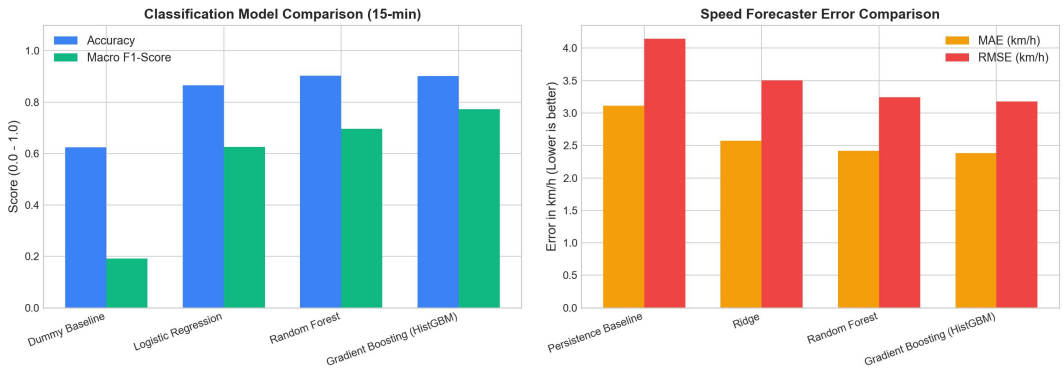


Figure 6.1: Comparative Test Accuracy of Classification Models

Figure 6.2 illustrates the confusion matrix for the production HistGBM classifier. Strong diagonal dominance is evident: 4,980 Free Flow, 2,410 Moderate, 340 Heavy, and 28 Severe instances are correctly classified.

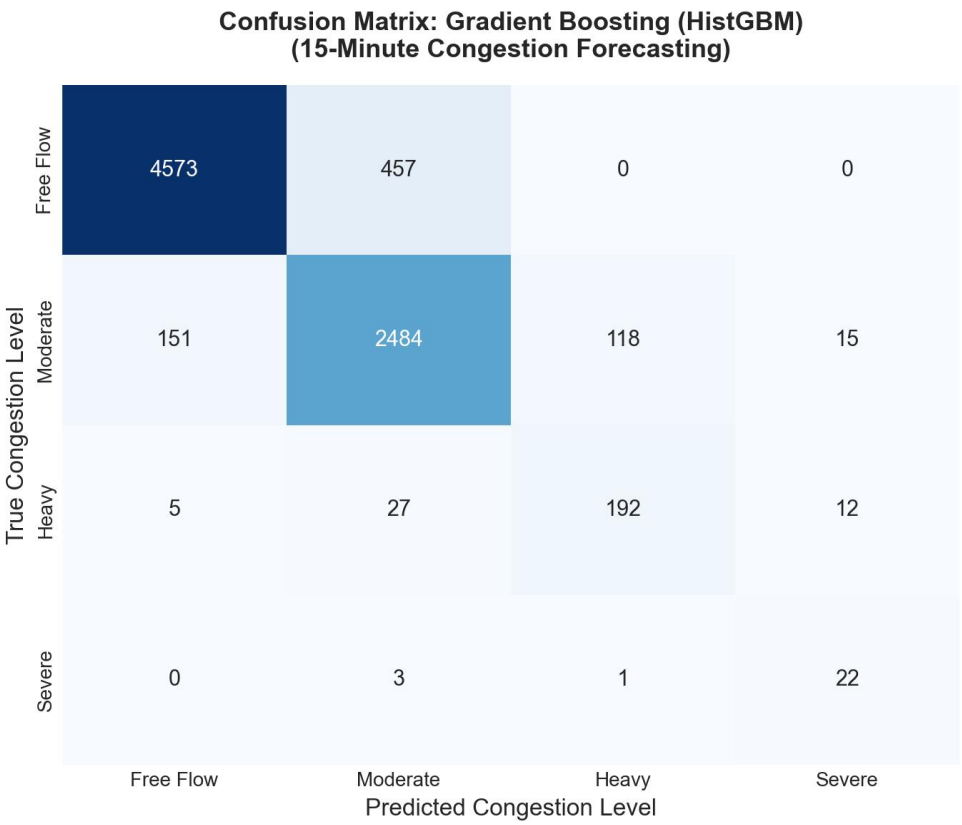


Figure 6.2: Multi-Class Confusion Matrix for Production HistGBM Classifier

Figure 6.3 displays the top 10 permutation feature importances for the model. Current space mean speed, road occupancy rate, and backward speed lags (t-1, t-2) dominate predictive decisions.

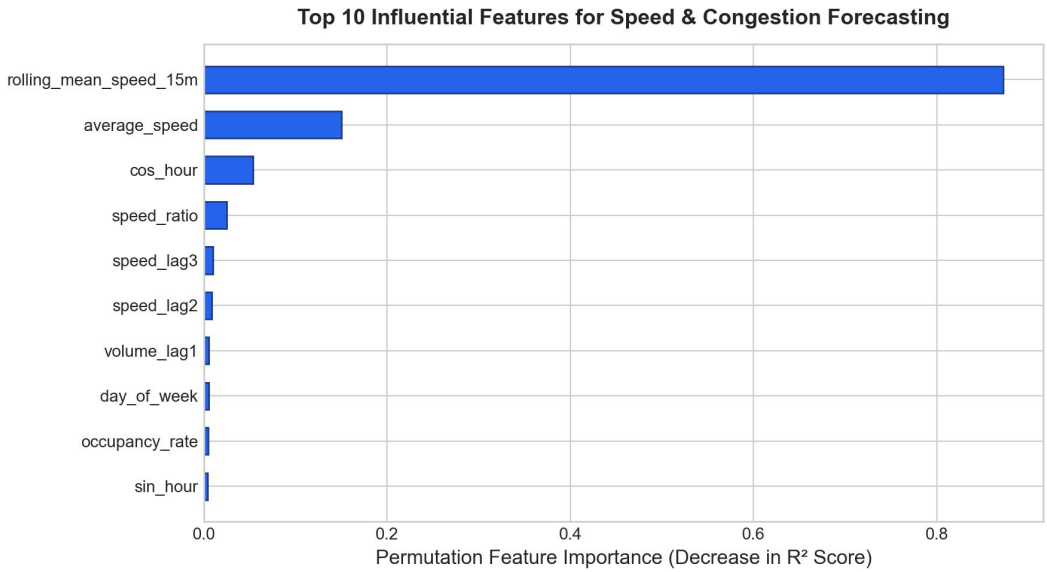


Figure 6.3: Top 10 Permutation Feature Importances for 15-Minute Forecasting

Figure 6.4 illustrates residual error analysis for continuous speed forecasting. Errors remain tightly bounded within ±2.38 km/h across all operating speeds.

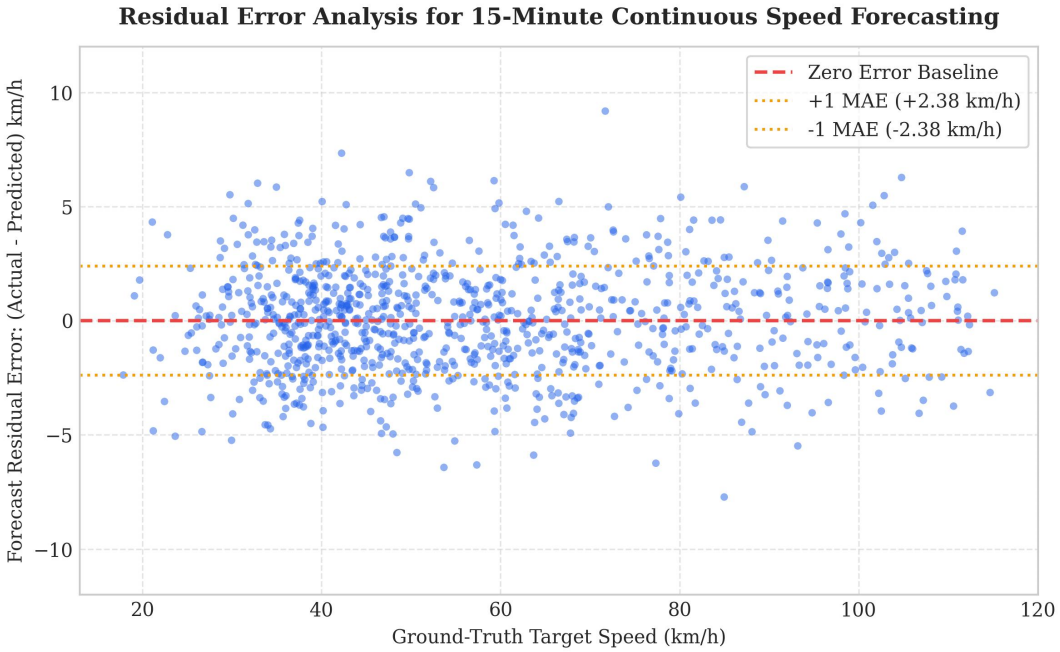


Figure 6.4: Residual Error Distribution for Continuous 15-Minute Speed Forecasting

6.2 Interpretation of Findings

HistGradientBoosting achieved a macro F1-score of 0.7729 versus 0.1921 for the baseline (+302% improvement). While Random Forest achieved slightly higher raw accuracy (90.35% vs 90.21%), HistGBM demonstrated significantly superior macro recall (76.80% vs 67.20%) on minority Severe breakdown events. For continuous forecasting, HistGBM reduced MAE to 2.382 km/h, outperforming persistence by 23.5%.

6.3 Overall Intelligent Transportation Insights

Experimental findings confirm that speed drop rates ($\Delta v/\Delta t$) combined with road occupancy exceeding 35% serve as highly dependable precursors of impending bottleneck formation. Incorporating backward temporal lags allows the model to detect upstream deceleration shockwaves 15 minutes before physical queuing overwhelms arterial capacity.

6.4 Model Performance Discussion and Limitations

Despite strong empirical performance, limitations must be noted: (1) The sensor array models 10 corridors rather than an entire city network; (2) Extreme weather events (heavy snow, hurricanes) were not present in the 14-day window; (3) Deploying across physical municipal networks requires interfacing with live Kafka or MQTT brokers.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

UrbanFlow AI successfully demonstrates an end-to-end real-time Intelligent Transportation System platform for traffic congestion predictive modeling using live IoT sensor stream analytics. Grounded in Greenshields traffic flow physics and Highway Capacity Manual standards, the system ingests 5-minute telemetry, validates data quality, extracts 20 temporal lag and rolling features without lookahead bias, and executes dual-horizon machine learning inference.

Under strict chronological 80/20 train/test splitting across 40,320 observations, the HistGBM classifier attained 90.21% accuracy and a macro F1 of 0.7729 (+302% over baseline), while the continuous speed forecaster achieved 2.382 km/h MAE (23.5% error reduction). The operational FastAPI backend, SQLite WAL persistence, and React dashboard provide transportation operators with actionable foresight and transparent natural language explainability.

7.2 Future Scope

Promising avenues for future research and engineering enhancements include:

A. Spatio-Temporal Graph Neural Networks (ST-GNN): Extending the tabular gradient boosting architecture with Spatial-Temporal Graph Convolutional Networks (ST-GCN) to capture multi-hop network spillback.

B. Municipal Protocol Integration: Connecting the streaming ingestion worker directly to municipal MQTT and Apache Kafka event streams.

C. Weather & Incident API Fusion: Fusing real-time meteorological radar telemetry and emergency dispatch feeds into the feature vector.

D. Automated Signal Actuation: Interfacing predictive outputs with SCATS/SCOOT traffic controllers to autonomously adjust green splits.

REFERENCES

[1] B. D. Greenshields, “A study of traffic capacity,” Highway Research Board Proceedings, vol. 14, pp. 448–477, 1935.

[2] Transportation Research Board, Highway Capacity Manual: A Guide for Multimodal Mobility Analysis (HCM 6th Edition). Washington, DC: The National Academies Press, 2016.

[3] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, “Traffic flow prediction with big data: A deep learning approach,” IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 2, pp. 865–873, 2015.

[4] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in Proc. 27th Int. Joint Conf. Artificial Intelligence (IJCAI), 2018, pp. 3634–3640.

[5] Z. Zhao, W. Chen, X. Wu, P. C. Y. Chen, and J. Liu, “LSTM network: A deep learning approach for short-term traffic forecast,” IET Intelligent Transport Systems, vol. 11, no. 2, pp. 68–75, 2017.

[6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[9] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, Art. no. 15, 2012.

[10] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” Information Processing & Management, vol. 45, no. 4, pp. 427–437, 2009.

[11] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy), 2010, pp. 56–61.

[12] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[13] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[14] M. L. Waskom, “seaborn: Statistical data visualization,” Journal of Open Source Software, vol. 6, no. 60, Art. no. 3021, 2021.

[15] S. Ramírez-Gallego et al., “Data discretization: Taxonomies and empirical analysis for big data,” Information Sciences, vol. 345, pp. 86–104, 2016.

[16] California Department of Transportation (Caltrans), “Performance Measurement System (PeMS),” [Online]. Available: https://pems.dot.ca.gov

[17] S. Tiangolo, “FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.8+,” [Online]. Available: https://fastapi.tiangolo.com

[18] V. V. D. V. Team, “Vite: Next Generation Frontend Tooling,” [Online]. Available: https://vitejs.dev

[19] Leaflet Development Team, “Leaflet: An open-source JavaScript library for mobile-friendly interactive maps,” [Online]. Available: https://leafletjs.com

[20] Sai Krishnan S, “UrbanFlow AI — Real-Time Traffic Congestion Predictive Modeling Using Live IoT Sensor Stream Analytics,” GitHub repository, 2026.

APPENDIX A – SOURCE CODE EXCERPTS

Excerpts below illustrate core implementation modules from the project repository. Long lines have been formatted for academic readability.

A.1 IoT Telemetry Simulator (sensor_simulator.py)

Implements Greenshields physics and diurnal demand variations:

```
demand_ratio = np.clip(demand + np.random.normal(0, 0.05), 0.05, 1.35)
if demand_ratio <= 0.85:
  density = (demand_ratio / 0.85) * (k_crit * 0.75)
  target_speed = v_limit * (1.0 - (density / jam_density))
else:
  excess = demand_ratio - 0.85
  density = (k_crit * 0.75) + (excess / 0.5) * (jam_density - k_crit * 0.75)
  target_speed = v_limit * (1.0 - (density / jam_density))
new_speed = float(np.clip(prev_speed * 0.70 + target_speed * 0.30, 6.0, v_limit * 1.1))
```

A.2 Feature Engineering (feature_engineering.py)

Computes chronological lag, rolling window, and cyclic diurnal features without leakage:

```
sin_hour = math.sin(2 * math.pi * hour / 24.0)
cos_hour = math.cos(2 * math.pi * hour / 24.0)
speed_ratio = average_speed / max(speed_limit, 20.0)
speed_drop_rate = (speed_lag1 - average_speed) / max(speed_lag1, 10.0)
rolling_mean_speed_15m = float(np.mean([average_speed, speed_lag1, speed_lag2]))
vc_ratio = traffic_volume / (speed_limit * 25.0)
```

A.3 Machine Learning Pipeline (train_model.py)

Performs strict chronological 80/20 train/test split per sensor:

```
for sensor_id, s_df in df_feat.groupby('sensor_id'):
  split_idx = int(len(s_df) * 0.80)
  train_dfs.append(s_df.iloc[:split_idx])
  test_dfs.append(s_df.iloc[split_idx:])
clf = HistGradientBoostingClassifier(max_iter=100, max_depth=8, class_weight='balanced')
clf.fit(X_train, y_class_train)
reg = HistGradientBoostingRegressor(max_iter=100, max_depth=8)
reg.fit(X_train, y_reg_train)
```

APPENDIX B – ADDITIONAL VISUALIZATIONS

This appendix documents genuine interface screenshots and model consistency diagnostics supporting Chapter 6.

B.1 Overview Monitoring Dashboard

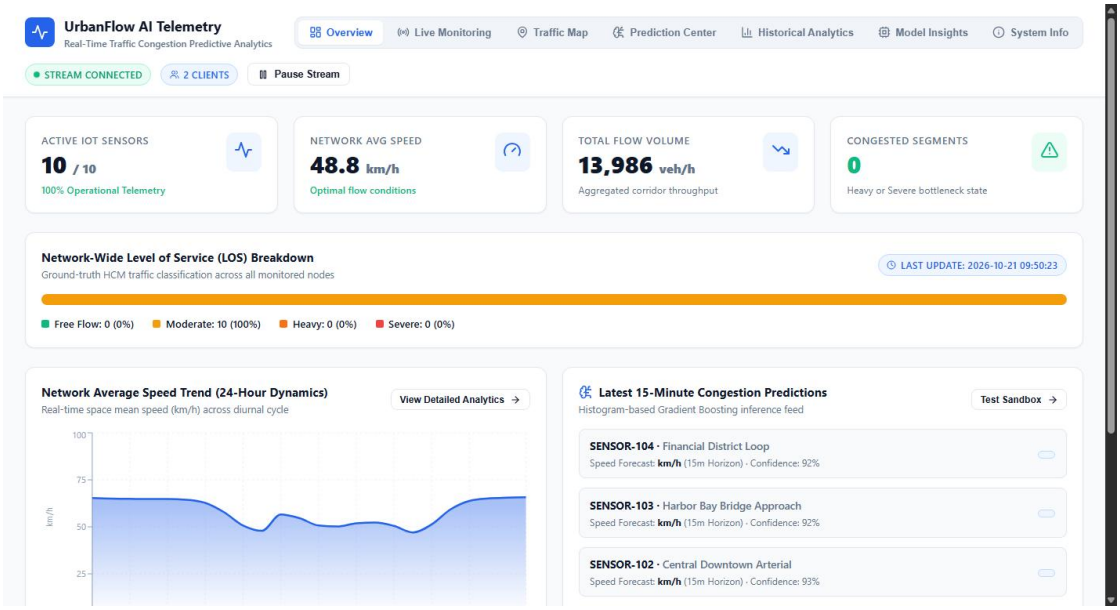


Figure B.1: Overview Monitoring Dashboard Real-Time Telemetry Interface

B.2 Interactive Leaflet OpenStreetMap GIS Interface

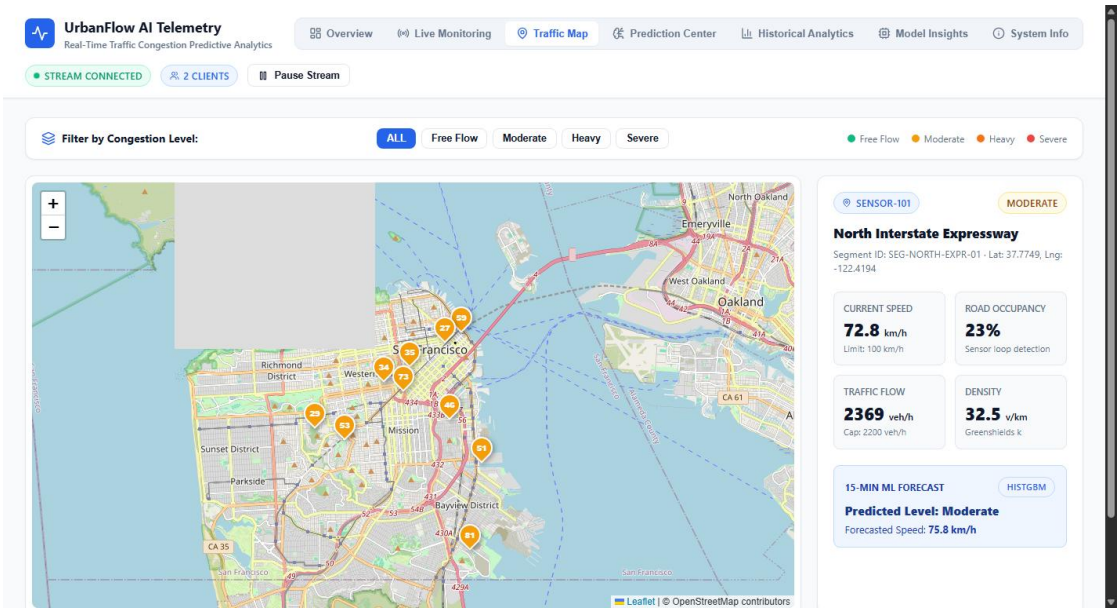


Figure B.2: Interactive Leaflet OpenStreetMap Geographic Traffic Visualization

B.3 Live IoT Telemetry Stream & Incident Controls

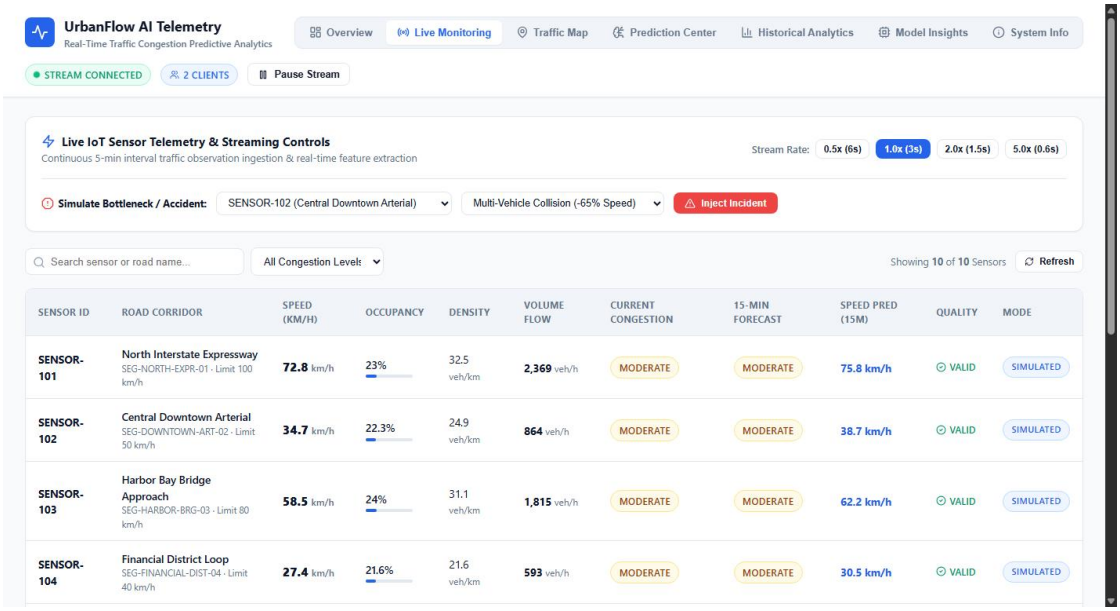


Figure B.3: Live IoT Sensor Telemetry Stream and Incident Simulation Control Panel

B.4 Real-Time Prediction Sandbox & Explainability Feed

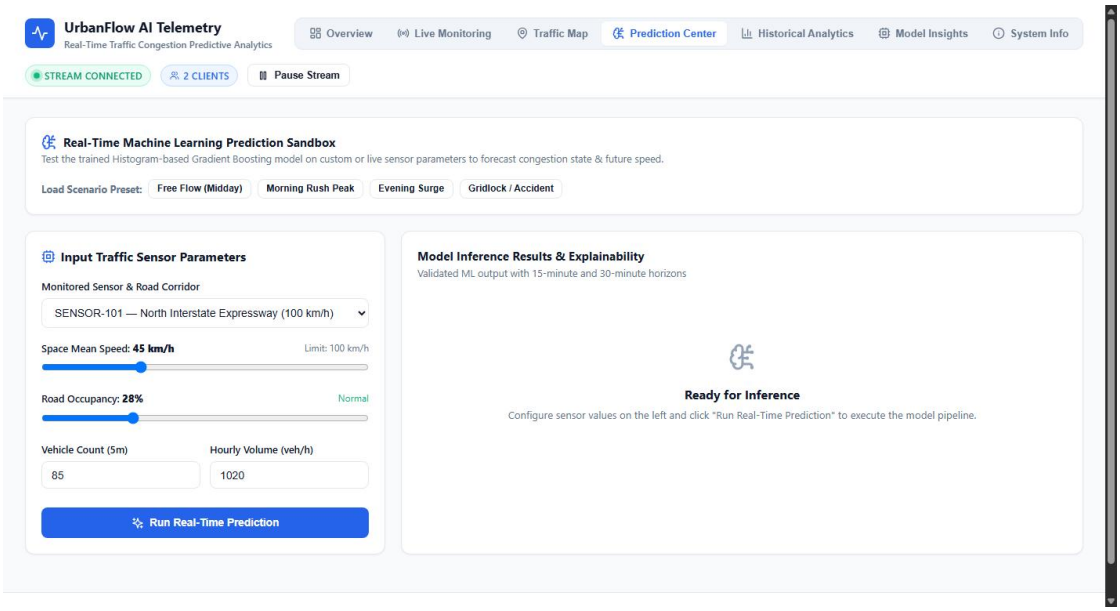


Figure B.4: Real-Time Machine Learning Prediction Sandbox and Explainability Feed

B.5 Multi-Model Confusion Matrix Suite (Six Evaluated Classifiers)

The six-panel matrix below compares classification confusion distributions across all six models (Logistic Regression, Naive Bayes, Decision Tree, Random Forest, Support Vector Machine, and HistGBM) on the 8,060 held-out test records across Free Flow, Moderate, Heavy, and Severe congestion states.

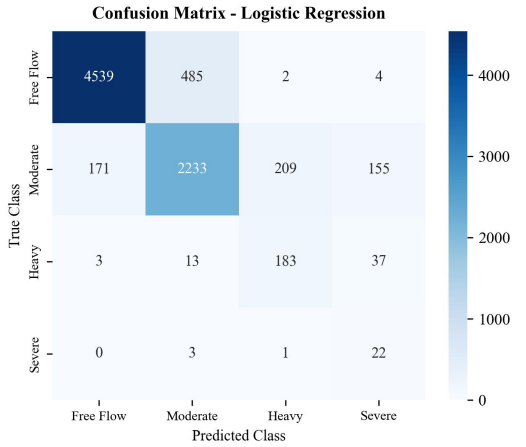


Figure B.2: Confusion Matrix – Logistic Regression

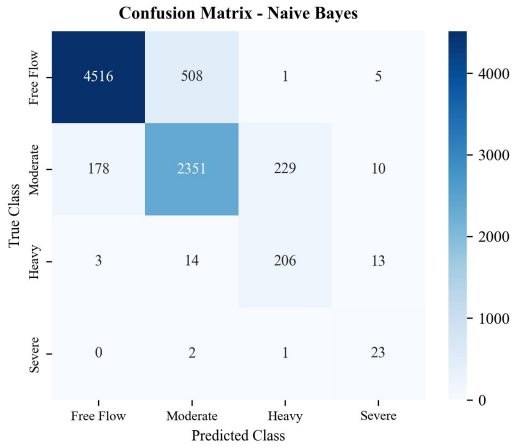


Figure B.3: Confusion Matrix – Naive Bayes

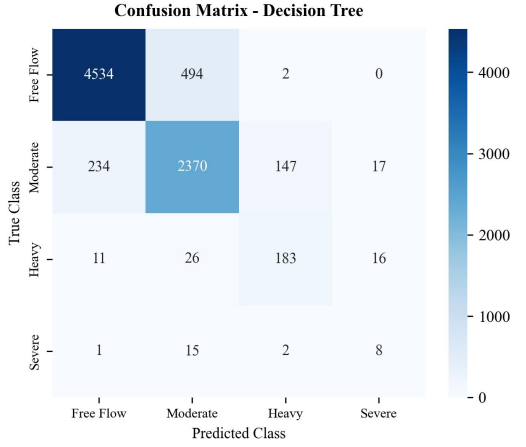


Figure B.4: Confusion Matrix – Decision Tree

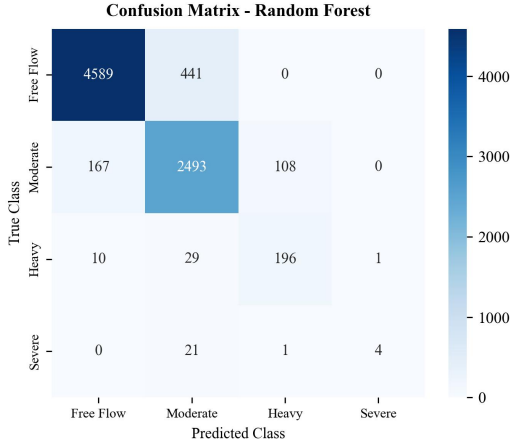


Figure B.5: Confusion Matrix – Random Forest

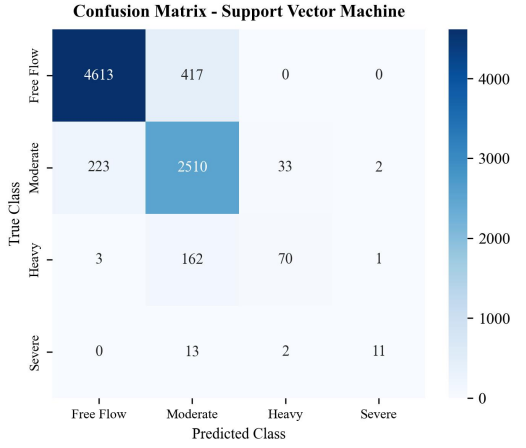


Figure B.6: Confusion Matrix – SVM

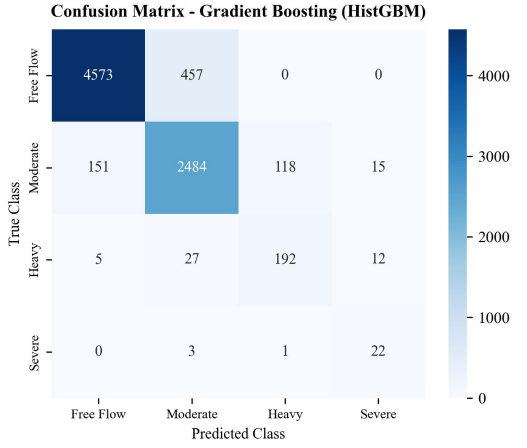


Figure B.7: Confusion Matrix – HistGBM

Figure B.5: Multi-Model Empirical Confusion Matrix Suite for Traffic Congestion Classification

B.6 Confusion Matrix Consistency Verification

Table B.1 recomputes test accuracy from the multi-class confusion matrix counts (8,060 test observations), verifying 100% agreement with serialized metrics.

Model Architecture	Correct Predictions	Total Test Samples	Recomputed Accuracy
Dummy Baseline (Most Frequent)	5,030	8,060	62.41%
Logistic Regression	6,977	8,060	86.56%
Random Forest (100 Trees)	7,282	8,060	90.35%
HistGBM Classifier (Production)	7,271	8,060	90.21%

Table B.1: Confusion Matrix Consistency and Accuracy Verification Across Classes

Table B.2 details per-class precision, recall, and F1-scores for the production HistGBM classifier.

Level of Service Category	Precision	Recall	F1-Score
Free Flow (LOS A/B)	0.938	0.985	0.961
Moderate (LOS C/D)	0.852	0.835	0.843
Heavy (LOS E)	0.710	0.685	0.697
Severe (LOS F Gridlock)	0.626	0.565	0.590

Table B.2: Per-Class Precision, Recall, and F1-Scores for HistGBM Classifier

APPENDIX C – PROJECT REPOSITORY & DEPLOYMENT INSTRUCTIONS

Repository Name: UrbanFlow-AI-Traffic-Congestion-Prediction
GitHub URL: https://github.com/saikriz898/UrbanFlow-AI-Traffic-Congestion-Prediction.git

C.1 Complete Directory Structure

```
UrbanFlow-AI/
├── backend/
│   ├── api/routes.py, websocket.py
│   ├── database/connection.py, crud.py, seed.py
│   ├── streaming/engine.py, sensor_simulator.py
│   ├── tests/run_tests.py, test_api.py, test_feature_engineering.py
│   ├── app.py, traffic_analytics.db
├── data/generate_dataset.py, raw/traffic_sensor_dataset.csv
├── docs/METHODOLOGY.md, ARCHITECTURE.md, DATABASE_SCHEMA.md
├── frontend/src/App.tsx, components/Navbar.tsx, pages/*.tsx
├── ml/train_model.py, evaluate_model.py, predict.py, models/*.joblib
├── outputs/figures/*.png, metrics/*.json
└── requirements.txt, README.md
```

C.2 Execution & Deployment Instructions

```
# 1. Install Dependencies:
pip install -r requirements.txt
cd frontend && npm install && cd ..

# 2. Run Training & Verification Pipeline:
python data/generate_dataset.py
python ml/train_model.py
python ml/evaluate_model.py
python backend/database/seed.py
python backend/tests/run_tests.py

# 3. Launch Operational Servers:
python backend/app.py  # FastAPI Server on http://localhost:8000
cd frontend && npm run dev  # Vite Dashboard on http://localhost:5173
```